Encapsulation:-

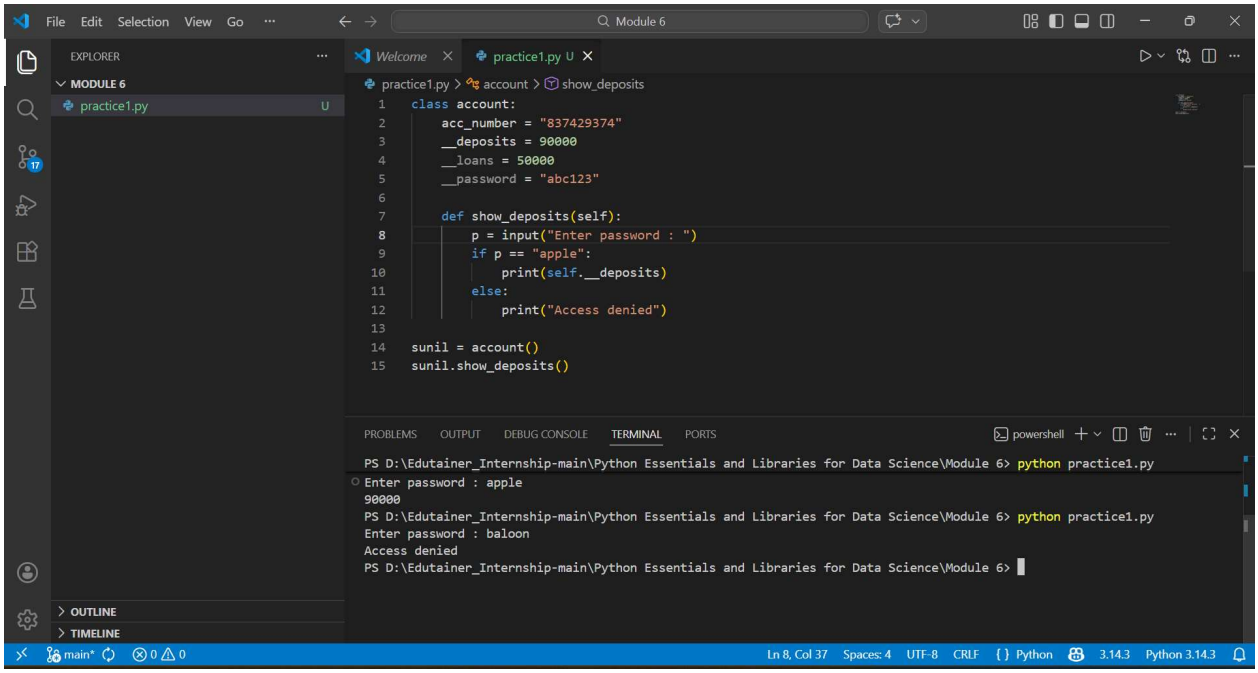
Encapsulation means wrapping data and methods together inside a class and restricting direct access to the internal data.

Suppose a bank maintains data about its customers such as:

- Deposits
- Loans
- Investments, etc.

If any outside entity wants to access this information, it must identify itself by providing passwords or signatures.

Hence, the customer data is protected (encapsulated) from the outside world.



Inheritance:-

Inheritance is the process by which one class acquires the properties and methods of another class.

A company employs different types of employees. However, every employee may have some data that is common and some data that is specific to that employee type.

Data which is common to all can be placed in a base class, and from the base class other subclasses can be derived.

Example Structure

Base Class:

Employee

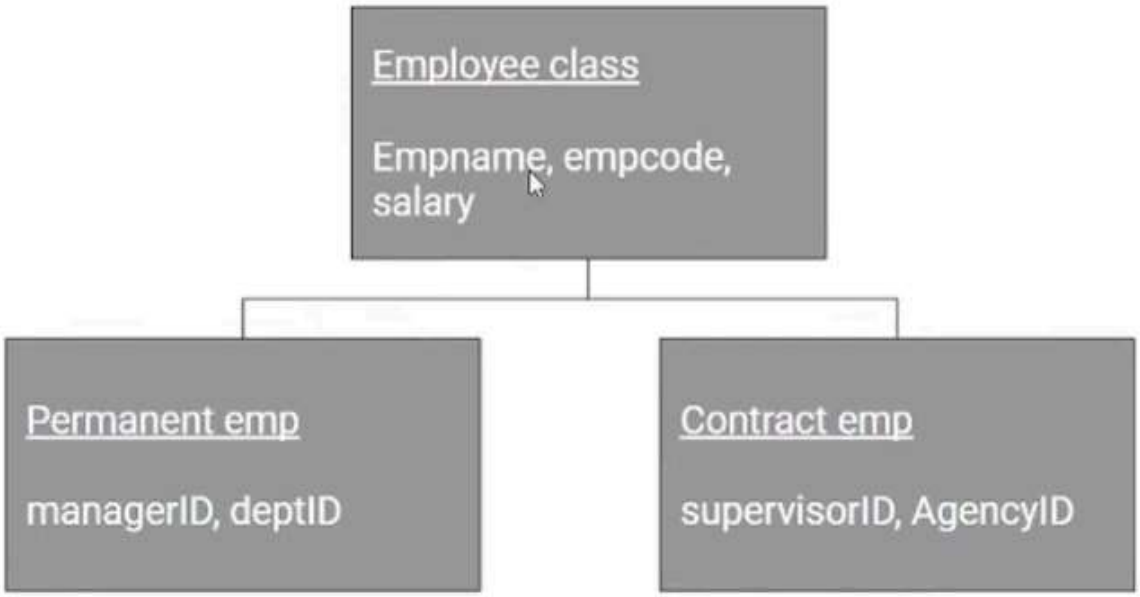
Subclasses:

Permanent Employee

Contract Employee

This means:

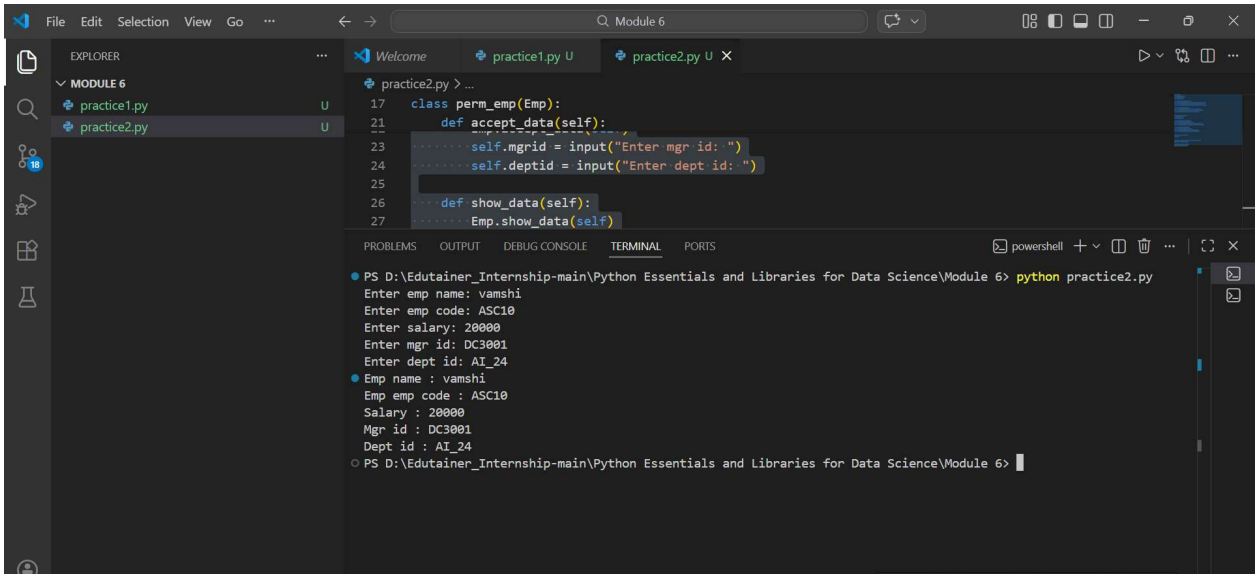
The Employee class contains common data (like name, id, salary)The Permanent and Contract classes inherit this data and may add their own specific details.




```
class Emp:
    emp_name = ""
    empcode = ""
    salary = 0

    def accept_data(self):
        self.emp_name = input("Enter emp name: ")
```

```
        self.empcode = input("Enter emp code: ")
        self.salary = int(input("Enter salary: "))

    def show_data(self):
        print("Emp name :", self.emp_name)
        print("Emp emp code :", self.empcode)
        print("Salary :", self.salary)


class perm_emp(Emp):
    mgrid = ""
    deptid = ""

    def accept_data(self):
        Emp.accept_data(self)
        self.mgrid = input("Enter mgr id: ")
        self.deptid = input("Enter dept id: ")

    def show_data(self):
        Emp.show_data(self)
        print("Mgr id :", self.mgrid)
        print("Dept id :", self.deptid)


class contr_emp(Emp):
    pass


# create object of the permanent employee
employee1 = perm_emp()

employee1.accept_data()
employee1.show_data()
```

Output:-



Polymorphism:-

Polymorphism means one method behaving differently for different objects or classes.

In a factory, there may be different units performing production and packing tasks.

The Factory class contains methods:

- production()
- packing()

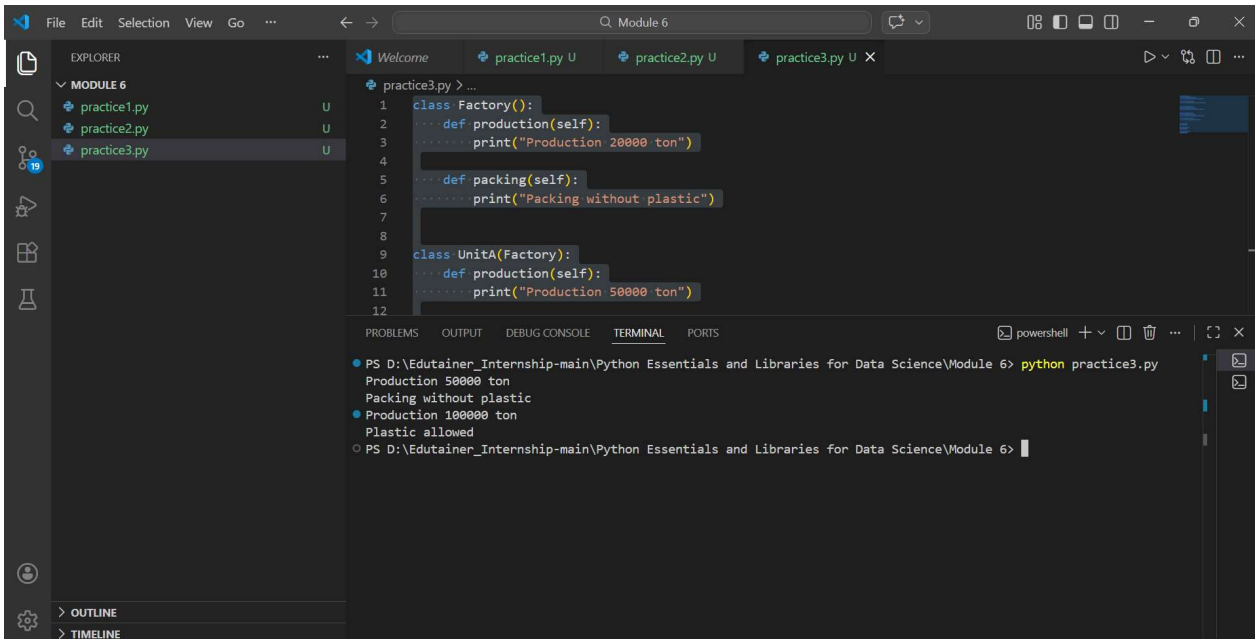
Two subclasses are derived from it:

- UnitA
- UnitB

Both units may perform production, but UnitB overrides the packing() method.Therefore, the packing() method becomes polymorphic.

```
class Factory():

    def production(self):

        print("Production 20000 ton")
```

```
    def packing(self):
        print("Packing without plastic")



class UnitA(Factory):
    def production(self):
        print("Production 50000 ton")



class UnitB(Factory):
    def production(self):
        print("Production 100000 ton")

    def packing(self):
        print("Plastic allowed")



ua = UnitA()
ub = UnitB()

ua.production()
ua.packing()
```

```
ub.production()

ub.packing()
```

Output:-